

אינטרפולציה

למה צריך אינטרפולציה? ברוב המקרים יש לנו $f(x)$ אלא שחשוב ערך של f הינו "יקר" יחסית, במיוחד אם צריך לחשב אותו הרבה פעמים (מיליוני פעמים). במקרה שכזה כדאי להכין מראש טבלה של ערכי f : $\{f_i\} = \{f(x_i)\}$ ואז להשתמש בטבלה על-ידי אינטרפולציה.

לדוגמה, משוואת מצב של חומר באנטרופיה קבועה. עבור גז אידיאלי $p_{eos} = p_0 \cdot (\rho / \rho_0)^\gamma$ כאשר p - לחץ, ρ - צפיפות ו γ - קבוע. במקרה הכללי, עבור חומרים שאינם גז אידיאלי, γ אינו קבוע והוא תלוי בצפיפות. במצב בו חישוב הלחץ הינו מורכב, ניתן לחסוך זמן ולבצע אינטרפולציה מטבלה שהוכנה מראש. לפעמים ניתן "לחסוך" ולקבל רמת דיוק נתונה עם טבלה דלילה יחסית, על-ידי בחירה נבונה של הגודל עליו מבצעים אינטרפולציה. במקרה שלנו, אם γ משתנה בצורה מתונה יותר מהלחץ, כדאי לשמור טבלה של γ . מקרה נוסף הוא מצב בו אנחנו יודעים רק ערכים בדידים של ומעוניינים בערכי ביניים (לדוגמא, מחיר החשמל עולה במהלך החודש והחשבון הסופי מחושב על-ידי ההערכה של הצריכה היחסית במהלך החודש). אינטרפולציה של פונקציה מתוך טבלה.

נתונה טבלה $X_0, X_1, \dots, X_n$ המתארת את ערכי $f = y(x)$ כאוסף בדיד של נקודות. יש למצוא את $y(x)$ $Y_0, Y_1, \dots, Y_n$

(כלומר להעריך את ערך הפונקציה) כאשר $x_0 \leq x \leq x_n$

האינטרפולציה מבוססת על פיתוח לטור טיילור סביב x ונעשית בשני שלבים:

1. מציאת i כך ש $x_{i-1} \leq x < x_i$

2. אינטרפולציה של $y(x)$ בתחום זה.

נתחיל עם השלב הראשון ונציג שלוש שיטות לחיפוש בטבלה מסודרת:

1. בטבלה קטנה : ניתן לעשות חיפוש brute force על כל האינדקסים עד שנמצא את הרלוונטי ב- $O(n)$ פעולות:

```
import sys
x = np.arange(0.,11.,1.)
x[1:-1] = x[1:-1] + (2.*np.random.rand(x.size-2)/3. - 1./3.)
xx = 10*np.random.rand()
for i in range(x.size):
    if x[i] > xx:
        break
else:
    print('ERROR',xx,x[0],x[-1])
    sys.exit('ERROR')
print(i,xx,x[i-1],x[i])
```

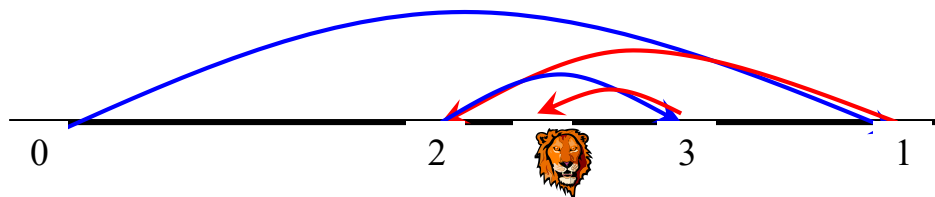
2. בטבלה עם מרווחים שווים ניתן לחשב ישירות את האינדקס:

```
x = np.arange(0.,11.,1.)
xx = 10*np.random.rand()
i = int((xx-x[0])/(x[1]-x[0])) + 1
print(i,xx,x[i-1],x[i])
```

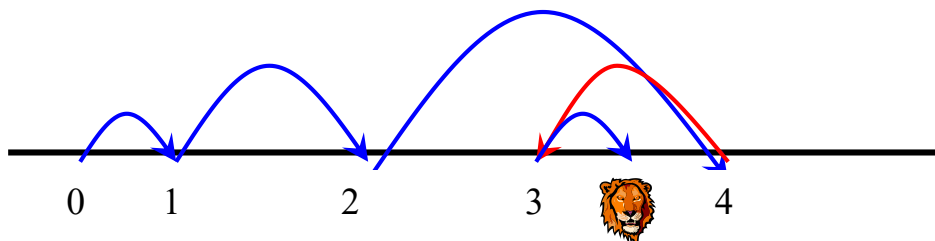
3. בטבלה כללית – "ציד אריות" הדורש $O(\log_2 n)$ פעולות במוצק:

```
x = np.arange(0.,11.,1.)
x[1:-1] = x[1:-1] + (2.*np.random.rand(x.size-2)/3. - 1./3.)
xx = 10*np.random.rand()
i_min, i_max = 0, x.size
while i_max-i_min > 1:
    i_av = (i_min + i_max) // 2
    if x[i_av] > xx:
        i_max = i_av
    else:
        i_min = i_av
i = i_max
print(i,xx,x[i-1],x[i])
```

הצגה גרפית של "ציד אריות"



כאשר מתחילים מניחוש קיים ניתן לשפר את החיפוש (במיוחד אם במוצק הניחוש ההתחלתי קרוב):



כעת נעבור לשלב השני של ביצוע האינטרפולציה לאחר שנמצא המרווח בו הנקודה נמצאת:
אינטרפולציה ליניארית (בין שתי נקודות): נגדיר את המרחקים בין הנקודות -

$$\begin{array}{ccc} h = x - x_{i-1} & k = x_i - x & \\ x_{i-1} & x & x_i \\ y_{i-1} & y & y_i \end{array}$$

פיתוח טיילור סביב x עבור y_i ועבור y_{i-1} :

$$\begin{aligned} y_i &= y + k \cdot y' + \frac{k^2}{2} \cdot y'' + \dots \\ y_{i-1} &= y - h \cdot y' + \frac{h^2}{2} \cdot y'' + \dots \end{aligned}$$

מתוך הביטוי ל $h \cdot y_i + k \cdot y_{i-1}$

$$y = \frac{h \cdot y_i + k \cdot y_{i-1}}{h + k} - \frac{h \cdot k}{2} \cdot y'' + \dots = \frac{(x - x_{i-1}) \cdot y_i + (x_i - x) \cdot y_{i-1}}{x_i - x_{i-1}} + O([h + k]^2)$$

נקבל:

בצורה דומה ניתן לפתח נוסחת אינטרפולציה המבוססת על 4 נקודות (Lagrange):

$$\begin{aligned} y_{i+1} &= y + (x_{i+1} - x) \cdot y' + \frac{(x_{i+1} - x)^2}{2} \cdot y'' + \frac{(x_{i+1} - x)^3}{6} \cdot y''' + \frac{(x_{i+1} - x)^4}{24} \cdot y'''' + \dots \\ y_i &= y + (x_i - x) \cdot y' + \dots \\ y_{i-1} &= y + (x_{i-1} - x) \cdot y' + \dots \\ y_{i-2} &= y + (x_{i-2} - x) \cdot y' + \dots \end{aligned}$$

על ידי ארבעת משוואות אלו ניתן לסלק את האיברים עם y', y'', y''' ולקבל ביטוי ל y מתוך $y_{i-2}, y_{i-1}, y_i, y_{i+1}$ עם שגיאה $O(\Delta x^4)$. ביטוי זה הוא למעשה פולינום מסדר שלישי העובר דרך נקודות הטבלה. דרך אלטרנטיבית לקבלת ביטוי שכזה היא על-ידי כך שנעביר מראש פולינום מסדר שלישי שעובר דרך ארבעת הנקודות $(x_i, y_i) \ i = 1..4$: לצורך כך נבנה ארבעה פולינומים $f_i(x), i = 1..4$ כך ש:

$f_1(x) = \frac{(x-x_2) \cdot (x-x_3) \cdot (x-x_4)}{(x_1-x_2) \cdot (x_1-x_3) \cdot (x_1-x_4)}$ - מתאפס ב- x_2, x_3, x_4 ושווה לאחד ב- x_1 . בצורה דומה בונים

את f_2, f_3, f_4 : $f_i(x) = \frac{\prod_{j \neq i} (x - x_j)}{\prod_{j \neq i} (x_i - x_j)}$ ואז הפולינום ממעלה שלישית שעובר דרך ארבעת הנקודות הוא:

$$P(x) = \sum_{i=1}^4 f_i(x) \cdot y_i$$

אינטרפולציה זו נותנת פונקציה רציפה אך הנגזרת אינה רציפה.

שיטה זו לקבלת נוסחת האינטרפולציה מאפשרת הרחבה פשוטה למספר שונה של נקודות המוביל לסדר אחר של הפולינום.

אינטרפולציה של Hermit - אינטרפולציה בין שתי נקודות שעבורן נתונה גם הנגזרת. כלומר, מחפשים $y(x)$ כאשר $x_1 \leq x < x_2$ וידועים y_1, y_1', y_2, y_2' . בונים ארבעה פולינומים ממעלה שלישית - f_1, f_2, g_1, g_2 שמקיימים:

$$\begin{aligned} f_1(x_1) &= 1, f_1(x_2) = 0, f_1'(x_1) = 0, f_1'(x_2) = 0 \\ g_1(x_1) &= 0, g_1(x_2) = 0, g_1'(x_1) = 1, g_1'(x_2) = 0 \\ f_2(x_1) &= 0, f_2(x_2) = 1, f_2'(x_1) = 0, f_2'(x_2) = 0 \\ g_2(x_1) &= 0, g_2(x_2) = 0, g_2'(x_1) = 0, g_2'(x_2) = 1 \end{aligned}$$

ואז פולינום האינטרפולציה הוא: $P(x) = \sum_{i=1}^2 f_i(x) \cdot y_i + \sum_{i=1}^2 g_i(x) \cdot y_i'$. באינטרפולציה זו גם הנגזרת

רציפה. ארבעה פולינומים שכאלה הם:

$$\begin{aligned} g_1(x) &= \frac{(x-x_1) \cdot (x-x_2)^2}{(x_1-x_2)^2}, \quad g_2(x) = \frac{(x-x_1)^2 \cdot (x-x_2)}{(x_1-x_2)^2} \\ f_1(x) &= \frac{(x-x_2)^2}{(x_1-x_2)^2} - \frac{2 \cdot g_1(x)}{x_1-x_2}, \quad f_2(x) = \frac{(x-x_1)^2}{(x_2-x_1)^2} - \frac{2 \cdot g_2(x)}{x_2-x_1} \end{aligned}$$

אינטרפולציית cubic spline

נניח שקיימת טבלה $y_i = y(x_i) \quad i = 1..N$ כפי שראינו, אינטרפולציה ליניארית באינטרוול $[x_j, x_{j+1}]$ היא

$$(\otimes) \quad y = A \cdot y_j + B \cdot y_{j+1}, \quad A = \frac{x_{j+1} - x}{x_{j+1} - x_j}, \quad B = \frac{x - x_j}{x_{j+1} - x_j}$$

ברור שבאינטרפולציה שכזו הנגזרת השנייה מתאפסת בתוך הקטעים והיא אינה מוגדרת בנקודות הקצה x_j .

המטרה ב-cubic spline היא אינטרפולציה בה הנגזרת הראשונה חלקה, והנגזרת השנייה רציפה. לשם כך נניח

שבנוסף לערכי y_j אנו יודעים גם את ערכי הנגזרת השנייה y_j'' , ואז נוסיף לנוסחת האינטרפולציה $(\otimes)$

פולינום ממעלה שלישית שהנגזרת השנייה שלו משתנה ליניארית בין y_j'' עבור x_j לבין y_{j+1}'' עבור x_{j+1} .

אם ערך הפולינום עצמו יתאפס בקצוות x_j, x_{j+1} הרי שההתאמה של y לא תתקלקל:

$$y = A \cdot y_j + B \cdot y_{j+1} + C \cdot y_j'' + D \cdot y_{j+1}''$$

$$C = \frac{1}{6} \cdot (A^3 - A) \cdot (x_{j+1} - x_j)^2, \quad D = \frac{1}{6} \cdot (B^3 - B) \cdot (x_{j+1} - x_j)^2$$

מכאן ניתן לבדוק את הנגזרות ולקבל:

$$(\oplus) \quad \frac{dy}{dx} = \frac{y_{j+1} - y_j}{x_{j+1} - x_j} - \frac{3 \cdot A^2 - 1}{6} \cdot (x_{j+1} - x_j) \cdot y_j'' + \frac{3 \cdot B^2 - 1}{6} \cdot (x_{j+1} - x_j) \cdot y_{j+1}''$$

$$\frac{d^2y}{dx^2} = A \cdot y_j'' + B \cdot y_{j+1}''$$

ואכן הנגזרת השנייה רציפה. אולם y_j'' אינו ידוע. הרעיון המרכזי הוא לדרוש שהנגזרת הראשונה תהיה רציפה

ולהשתמש ב- $(\oplus)$ עבור ערכו של y_j' פעם בקצה השמאלי של הקטע $[x_j, x_{j+1}]$ ופעם בקצה הימני של הקטע

$[x_{j-1}, x_j]$. מכאן נקבל עבור $j = 2..N - 1$:

$$\frac{x_j - x_{j-1}}{6} \cdot y_{j-1}'' + \frac{x_{j+1} - x_{j-1}}{3} \cdot y_j'' + \frac{x_{j+1} - x_j}{6} \cdot y_{j+1}'' = \frac{y_{j+1} - y_j}{x_{j+1} - x_j} - \frac{y_j - y_{j-1}}{x_j - x_{j-1}}$$

כלומר יש $N - 2$ משוואות ליניאריות לקביעת y_j'' , $j = 2..N - 1$. פותרים זאת על ידי קביעת y_1'', y_N''

(למשל איפוסם או קביעתם כך שלנגזרת הראשונה בקצוות יהיה ערך נתון). מערכת המשוואות שהתקבלה היא

מערכת טרידיאגונלית אותה קל לפתור (כפי שנלמד בשיעורים הבאים).

אינטרפולציה רב ממדית

לדוגמא: פונקציה של שני משתנים – משוואת מצב $p = p_{eos}(\rho, T)$ כאשר p - לחץ, ρ - צפיפות ו T -

טמפרטורה. ניתן לחשב טבלה דו-ממדית של ערכי הלחץ $\{p_{i,j}\}$ עבור ערכים

	T_1	T_2	T_3	
ρ_1	$p_{1,1}$	$p_{1,2}$	$p_{1,3}$	$\dots$
ρ_2	$p_{2,1}$	$p_{2,2}$	$p_{2,3}$	
ρ_3	$p_{3,1}$	$p_{3,2}$	$p_{3,3}$	
	$\vdots$			

בדידים של הצפיפות $\{\rho_i\}$ והטמפרטורה $\{T_j\}$. כאשר צריך להעריך את $p(\rho, T)$ נמצא i כך ש $\rho_{i-1} < \rho \leq \rho_i$ וכן j כך ש $T_{j-1} < T \leq T_j$ ואז

נשתמש בנוסחת אינטרפולציה בילינארית:

$$p(\rho, T) = (1 - \alpha) \cdot (1 - \beta) \cdot p_{i-1, j-1} + \alpha \cdot (1 - \beta) \cdot p_{i, j-1} + (1 - \alpha) \cdot \beta \cdot p_{i-1, j} + \alpha \cdot \beta \cdot p_{i, j}$$

כאשר $\alpha = \frac{\rho - \rho_{i-1}}{\rho_i - \rho_{i-1}}$, $\beta = \frac{T - T_{j-1}}{T_j - T_{j-1}}$ גם כאן ניתן לקבל נוסחאות אינטרפולציה עם פולינומים בסדרים

גבוהים יותר, הלוקחות בחשבון שכנים רחוקים יותר.